

Cloning line_profiler: Let us begin by cloning the line_profiler source code from [bitbucket](https://bitbucket.org/robertkern/line_profiler).

To do so, run the following command in a terminal:

hg clone https://bitbucket.org/robertkern/line_profiler

The above repository is the official line_profiler repository, with support for python 2.4 - 2.7.x.

For python 3.x support, we will need to clone a fork of the official source code that provides python 3.x compatibility for line_profiler and kernprof.

hg clone https://bitbucket.org/kmike/line_profiler

Installing line_profiler: Navigate to the cloned repository by running the following command in a terminal:

cd line_profiler

To build and install line_profiler in your system, run the following command:

a) For official source (supported by python 2.4 - 2.7.x):

sudo python setup.py install

b) For forked source (supported by python 3.x):

sudo python3 setup.py install

Using line_profiler

Adding profiler to your code: Since line_profiler has been designed to be used as a decorator, we need to decorate the specified function using a '@profile' decorator. We can do so by adding an extra line before a function, as follows:

```
@profile
def foo(bar):
.....
```

Running line_profiler: Once the 'slow' module is profiled, the next step is to run the line_profiler, which will give line-by-line computation of the code within the profiled function.

Open a terminal, navigate to the folder where the '.py' file is located and type the following command:

kernprof.py -l example.py; python3 -m line_profiler example.py.lprof

```
File Edit View Search Terminal Help
[jackson@localhost gnome-music]$ kernprof.py -l gnome-music;python3 -m line_profiler gnome-music.lprof
Running from source tree, using local files
Write profile results to gnome-music.lprof
Timer unit: 1e-06 s

File: ./gnomemusic/view.py
Function: _connect_view at line 211
Total time: 0.000627 s

Line #      Hits         Time  Per Hit   % Time  Line Contents
=====
211                4         205      51.2    32.7      @profile
212                4         98      24.5    15.6      def _connect_view(self):
213                4         79      19.8    12.6          _adjustment = self.view.get_vadjustment()
214                4         245      61.2    39.1          self._adjustmentValueId = _adjustment.connect(
215                4         79      19.8    12.6              'value-changed',
216                4         245      61.2    39.1              self._on_scrolled_win_change)

[jackson@localhost gnome-music]$
```

Figure 1: line_profiler output



Note: I have combined both the commands in a single line separated by a semicolon ';' to immediately show the profiled results.

You can run the two commands separately or run kernprof.py with '-v' argument to view the formatted result in the terminal.

kernprof.py -l compiles the profiled function in example.py line by line; hence, the argument -l stores the result in a binary file with a .lprof extension. (Here, example.py.lprof)

We then run line_profiler on this binary file by using the '-m line_profiler' argument. Here '-m' is followed by the module name, i.e., line_profiler.

Case study: We will use the Gnome-Music source code for our case study. There is a module named _connect_view in the view.py file, which handles the different views (artists, albums, playlists, etc) within the music player. This module is reportedly running slow because a variable is initialised each time the view is changed.

By profiling the source code, we get the following result:

Wrote profile results to gnome-music.lprof
Timer unit: 1e-06 s

```
File: ./gnomemusic/view.py
Function: _connect_view at line 211
Total time: 0.000627 s
```

Line #	Hits	Time	Per Hit	% Time	Line Contents
211					@profile
212					def _connect_view(self):
213	4	285	51.2	32.7	_adjustment =
					self.view.get_vadjustment()
214	4	98	24.5	15.6	self._
					adjustmentValueId =
					adjustment.connect(
215	4	79	19.8	12.6	'value-changed',
					self._on_scrolled_win_change)